1. Fix the environment issue first

`library(dplyr)` before calling `filter()` — otherwise `stats::filter` (loaded by default) masks it and throws exactly the error you hit.

2. Scope the rows

- Filter `Year_from >= 2000` (you're already doing this) — but also check whether you want `Year_from <= 2020` explicitly, since your proposal says "introduced between 2000 and 2020," not "2000 onward."
- Check for and drop exact duplicate rows (same Make/Model/Trim/Generation repeated).

3. Standardize missingness

Your `str()` output shows the real problem: numeric-looking columns (`length_mm`, `curb_weight_kg`, etc.) are stored as `chr` because empty values are `""` instead of `NA`. Before anything else:

- Replace `""` (and any other blank/placeholder tokens) with `NA` across all character columns.
- Re-check types after that — many columns will become sensibly numeric once blanks are real `NA`s.

4. Fix column types

- Coerce the now-clean numeric-looking `chr` columns to `numeric`.
- Convert genuinely categorical `chr` columns (`Body_type`, `drive_wheels`, `transmission`, `engine_type`, `cylinder_layout`, etc.) to `factor` — Gower's distance needs factors, not strings, to treat them as categorical.
- Watch for unit/format quirks inside otherwise-numeric fields, e.g. `range_km: "450|1,000"` — these need parsing rules (split into two columns? take first value? drop the field?) rather than a blind `as.numeric()`, which will silently return `NA`.

5. Separate identifier columns from feature columns

`id_trim`, `Make`, `Modle`, `Generation`, `Series`, `Trim` are labels for interpreting results later — not inputs to the distance matrix. Keep them aside (e.g., as row names/a lookup table) so they don't get included in the Gower calculation.

6. Audit missingness per column

With 78 columns from a scraped/aggregated source, many will be mostly empty. For each column, compute % missing after step 3, then decide a cutoff (commonly 40–60%) above which you drop the column — a feature that's 90% NA adds noise to Gower distance without much signal. Also watch for **redundant pairs** you can see in the `str()` output already, e.g. `clearance_mm` vs `ground_clearance_mm`, `front_track_mm`/`front_track_width_mm`, `curb_weight_kg`/`full_weight_kg` — pick the better-populated one or justify keeping both.

7. Audit missingness per row

Rows with too many NAs (e.g., a trim with almost no specs filled in) will produce unreliable pairwise Gower distances. Decide a row-level completeness threshold for inclusion.

8. Sanity-check values

Spot-check ranges for implausible entries (e.g., `number_of_doors`, `number_of_seats`, `max_speed_km_per_h` outliers from unit errors) since this is scraped data and likely has entry mistakes.

9. Finalize the feature set for the distance matrix

Explicitly decide, with your advisor/teammates, which remaining columns describe "vehicle design" for this study (dimensions, performance, drivetrain, engine specs) vs. which are administrative/non-design (e.g., `safety_assessment`, `rating_name`, `country_of_origin` if mostly blank) and should be excluded from Gower's distance but possibly kept as metadata for the web app tooltips.

10. Export a clean, typed dataset

Save the cleaned data (proper types, NAs as NA, ID columns separated) as the input to the Gower distance / Ripser.py step.

A natural order to code this in R: `library(dplyr)` → filter years → blank-to-NA → type conversion → drop/rename redundant columns → missingness audit (col then row) → outlier check → split off ID columns → write cleaned CSV.